Weight-Space Programming of Variational Quantum Circuits: Large Language Models Write Quantum Circuit Parameters Without an Optimizer

Debarshi Kundu[1], Swaroop Ghosh[1], and Vasant Honavar[1]

[1]The Pennsylvania State University

Abstract

Variational quantum algorithms train the angles of a parameterized quantum circuit with a classical optimizer, and the optimizer is the bottleneck: every loss evaluation is a run of the quantum device, gradients vanish for generic circuits, and the problem is NP-hard in general. We ask whether a large language model can write the angles directly. *Weight-space programming*, introduced by Takehi et al. for classical neural networks, has a general-purpose LLM read and write a network's weights directly, without an optimizer; we bring it to quantum circuits. The model is given the problem, the circuit and the gate conventions, and replies with the numbers; the circuit is simulated exactly as written, and no loss, gradient or accuracy is ever returned. We test a frontier model (GPT-6 Astra) on the three main families of variational algorithms: classification of handwritten digits with a quantum circuit, ground-state energies of spin chains and H_2 (VQE), and MaxCut (QAOA), 139 requests in all, each compared against random weights, a classical optimizer, and the optimizer started from the model's weights on the same circuit. Where the weights have a meaning the model can reason about, it matches the optimizer: 95–100% of the energy spectrum on every VQE problem, the exact H_2 ground state from a closed-form derivation, optimal depth-one QAOA angles on every graph, and 93–96% accuracy on 3 vs 8 from twelve hand-written weights of a fixed circuit, above an optimizer trained on the same ten examples. Its weights are also a strong warm start: the optimizer started from them reaches its best value after a median of one evaluation instead of hundreds. Left to design the circuit, the model chooses, in 30 of 30 cases, a single-qubit rotation chain whose weights are pixel importances, and programs it to 65% accuracy on all ten digits. The method does not scale to opaque weight spaces (accuracy falls to chance as a fixed circuit is deepened past 30 weights while the optimizer improves) and is sensitive to conventions (6 of 42 QAOA replies negate the mixing angles and collapse). We release every prompt, reply and score.

1 Introduction

Every variational quantum algorithm has the same shape as a neural network: a fixed computational graph (the circuit) with continuous parameters (the gate angles), a loss computed by running the graph, and a classical optimizer that adjusts the parameters [Cerezo et al., 2021]. The optimizer is the expensive part. Each loss evaluation is a run of the quantum device, gradients vanish exponentially in the number of qubits for generic circuits [McClean et al., 2018], and the optimization problem is NP-hard in general [Bittel and Kliesch, 2021]. A great deal of work therefore goes into choosing good initial parameters, or into reusing parameters across problem instances.

This paper asks a simple question: *can a large language model write the parameters directly?* Not propose a circuit for someone else to train, and not act as an optimizer that is shown scores and

asked to improve them, but read a description of the problem—a Hamiltonian, a graph, ten pictures of handwritten digits—and reply with the numbers that go into the gates, with no feedback of any kind. Takehi et al. [2026] recently showed that a frontier model can do this for classical neural networks: read the weights of a trained network and say what it does, edit them to fix a failing policy, and write the weights of game-playing networks from scratch. They call it *weight-space programming*—a general-purpose LLM reads and writes a network's weights directly, not through an optimizer, around a natural-language specification and sometimes with no environment feedback at all. Our work is inspired by theirs and carries the idea to a domain where it has a particular pull: variational quantum circuits have few parameters, each with a physical or geometric meaning, and the cost of the optimizer they replace is paid in runs of a quantum device.

We test the question on the three main families of variational quantum algorithms: supervised classification with a parameterized circuit (QML), ground-state energy estimation (VQE), and combinatorial optimization (QAOA). In every experiment the model receives the problem, the exact gate conventions and the circuit (or, in one QML setting, a small circuit language in which it may design its own), and returns a JSON list of angles. The angles are simulated exactly as written. We compare them against three references on the same circuit: random angles, a classical optimizer started from random angles, and the same optimizer started from the model's angles. All experiments use one frontier model, GPT-6 Astra, and every prompt, reply and score is released.

Findings.

1. **Weight programming works when the weights have meaning.** With the Hamiltonian variational ansatz for the transverse-field Ising chain, the model's angles capture 95–100% of the spectrum between the highest and lowest eigenvalue on every problem, equal to the optimizer; for H_2 it derives the energy as a function of the single angle and writes its exact minimizer; for QAOA at depth one it derives the closed-form expectation of the cut and maximizes it analytically, matching the optimizer on all six graphs. For digit classification it hand-writes 12–18 weights of a fixed amplitude-encoding circuit and scores 93–96% on 3 vs 8, *above* an optimizer trained on the same ten examples.

2. **The model's weights are a strong warm start.** Started from the model's angles, the optimizer reaches its best value after a median of one loss evaluation in VQE and QAOA, against hundreds from random starts, and it reaches that value more often.

3. **Left to design the circuit, the model picks the one it can reason about.** In the free-design QML setting the model converges, in 30 of 30 trials across six conditions, on a single-qubit chain of data-dependent rotations: a linear classifier in disguise, with hand-chosen pixel weights that score 91–95% on binary tasks and 65% on all ten digits. When structural rules forbid this, it composes two-qubit circuits with the same logic. Interpretability of the weight space, not quantumness, is what the model exploits.

4. **It does not scale to opaque weight spaces.** When the fixed circuit is deepened, so that each weight's effect on the output can no longer be reasoned about, accuracy on 3 vs 8 falls from 0.96 at 18 weights to 0.70 at 54, and on ten classes it never leaves chance (0.07–0.14) while the optimizer on the same examples climbs to 0.57. The failure is informative: the model says it is writing "heuristic weights" and it is right.

5. **Conventions are a failure mode.** In 6 of 42 QAOA trials the model produced the correct angle magnitudes with every mixer angle negated; those circuits score near zero, and with the

sign corrected they match the optimizer. Weight programming is only as good as the shared understanding of what a weight means.

These results extend weight-space programming from classical networks to quantum circuits and reframe what a language model can contribute to variational quantum algorithms: not as a replacement for the optimizer in general, but as a source of parameters for circuits whose weights carry physical or geometric meaning, and as an initializer that removes most of the optimizer's work.

2 Background for machine-learning readers

This section gives the minimum needed to read the rest of the paper. Readers familiar with variational quantum algorithms can skip to Section 3.

Qubits and states. A classical bit is 0 or 1. A *qubit* is described by two complex numbers (α, β) with $|\alpha|^2 + |\beta|^2 = 1$; measuring it yields 0 with probability $|\alpha|^2$ and 1 with probability $|\beta|^2$. A register of n qubits is described by 2^n complex numbers (the *amplitudes*), one per bit string, with squared magnitudes summing to one. Measuring the register returns one bit string, drawn with probability equal to its squared amplitude. All the circuits in this paper are small enough (≤ 8 qubits) that we compute these 2^n amplitudes exactly on a laptop, so every number we report is a noise-free expectation, not a sampled estimate.

Gates are the layers; angles are the weights. A quantum circuit is a sequence of *gates*, each a small unitary matrix applied to one, two, or three qubits. Some gates are fixed (the Hadamard gate h, the controlled-NOT cx), others are *rotations* with a continuous angle: $ry(\theta)$ rotates one qubit by θ about the Y axis of its state sphere, $rzz(\theta)$ couples two qubits. A circuit with free angles is a *parameterized quantum circuit* (PQC). The analogy to a neural network is close and worth keeping in mind: the gate layout is the architecture, the angles are the weights, and the measurement is the readout. Two angles on the same qubit about the same axis compose by addition, $ry(a)\,ry(b) = ry(a+b)$, exactly as two consecutive linear layers without a nonlinearity collapse to one; entangling gates such as cx play the role of the nonlinearity that keeps depth from collapsing.

Variational quantum algorithms. A variational quantum algorithm (VQA) [Cerezo et al., 2021] fixes a circuit layout, evaluates a loss by running the circuit and measuring, and updates the angles with a classical optimizer. The three families we study differ only in the loss:

- **Quantum machine learning (QML).** The input x is encoded into the circuit (as rotation angles proportional to pixel values, or as the amplitudes themselves), the angles are the trainable weights, and a measured probability is the class score [Mitarai et al., 2018, Schuld et al., 2020, Pérez-Salinas et al., 2020]. The loss is a classification loss on labelled data.
- **Variational quantum eigensolver (VQE).** The loss is the energy $\langle\psi(\theta)|H|\psi(\theta)\rangle$ of the prepared state under a Hamiltonian H (a Hermitian matrix describing a molecule or a magnet); the minimum over all states is the *ground-state energy* E_0 [Peruzzo et al., 2014]. H is written as a sum of Pauli terms, e.g., $-\sum_i Z_i Z_{i+1} - h\sum_i X_i$ for a chain of spins with nearest-neighbour coupling and a transverse field h.
- **Quantum approximate optimization algorithm (QAOA).** The loss is the negative expected value of a combinatorial objective, here the MaxCut value of a graph [Farhi et al.,

2014]. The circuit alternates p layers of a "cost" step $e^{-i\gamma_k C}$ and a "mixing" step $e^{-i\beta_k \sum_i X_i}$; the weights are the $2p$ angles $(\gamma_1, \beta_1, \ldots, \gamma_p, \beta_p)$.

Why the weights are the hard part. Training a VQA is, in general, NP-hard [Bittel and Kliesch, 2021], and generic circuits suffer from *barren plateaus*: for random initializations the gradient of the loss vanishes exponentially in the number of qubits [McClean et al., 2018], so an optimizer starting from random weights may learn nothing. Each loss evaluation is a run of the quantum device, so the number of evaluations an optimizer consumes is the dominant cost. Good initial weights are therefore valuable in their own right [Grant et al., 2019, Egger et al., 2021], and for QAOA on regular graphs it is known that near-optimal angles are nearly instance-independent [Brandão et al., 2018, Zhou et al., 2020, Wurtz and Love, 2021].

Two standard circuit layouts. The *hardware-efficient ansatz* (HEA) [Kandala et al., 2017] alternates a layer of single-qubit rotations with a ladder of *cx* gates; it is generic and says nothing about the problem. The *Hamiltonian variational ansatz* (HVA) [Wecker et al., 2015, Wiersema et al., 2020] builds layers from the terms of H itself, so each angle has a physical meaning (how long to evolve under the coupling term, how long under the field term). We use both, because they differ in how much the structure of the problem is visible in the structure of the weights.

3 Weight-space programming for quantum circuits

We follow the setting of Takehi et al. [2026]: a general-purpose model writes a network's weights directly, without an optimizer, from a natural-language specification. The network is a parameterized quantum circuit and the weights are its gate angles.

Protocol. A weight-space programming trial is a single request to a language model. The prompt contains (i) the problem—a Hamiltonian, a graph, or a handful of labelled images; (ii) the circuit, either fully specified with named angles $\theta[0..K-1]$, or a description of a small circuit language in which the model may design its own; (iii) the exact gate conventions (we use the Qiskit definitions, e.g., $ry(t) = e^{-itY/2}$); and (iv) the reply format, a JSON object with a list of K numbers (or a full circuit). The model's reply is simulated exactly as written. No loss value, gradient, or accuracy is ever sent back. The only feedback loop is syntactic: if the reply is not a valid circuit (wrong number of weights, a gate on a non-existent qubit, a malformed expression), the validation error is returned and the model may answer again, at most twice. Every prompt, reply, and parsed circuit is saved, and the model's free-text reasoning is kept alongside the numbers.

Two levels of freedom. In the *weights-only* setting the circuit is fixed and the model writes K real numbers. This is the clean test of the hypothesis, and it is the setting of all VQE and QAOA experiments and of the fixed-ansatz QML experiment. In the *free-design* setting (QML only) the model also chooses the number of qubits, the data encoding, the gates, and the readout, within a small circuit language described in Appendix A; this tests whether the model can invent a usable architecture and program it in one shot.

What "works" means. Numbers written by a model are only interesting relative to what the same circuit can do. For every trial we therefore evaluate, on the same circuit layout:

- **Random weights.** The same layout with every angle drawn uniformly from $[-\pi, \pi]$ (50 draws for VQE/QAOA, 20 for QML). This is the floor: it tells us how much of the score comes from the layout and the readout rather than from the numbers.
- **Optimizer.** A classical optimizer (L-BFGS-B with finite-difference gradients) started from random weights, with five restarts for VQE/QAOA and one for QML. For QML it sees exactly the examples the model saw, so it is the fair "same information, numerical optimization instead of reasoning" comparison. This is the ceiling the layout admits under modest effort, not a tuned upper bound.
- **Optimizer from the model's weights.** The same optimizer started where the model left off. If the model's weights are near a good basin the optimizer should reach the optimum quickly; we count the number of loss evaluations it needs to come within 0.1% of the best value found from random starts.
- **Exact and classical references.** The exact ground energy and the brute-force maximum cut, computed by diagonalization and enumeration; and, for QML, logistic regression on the raw pixels.

For energies we report the fraction of the spectrum captured, $r = (E_{\max} - E)/(E_{\max} - E_0)$, which is 1 at the ground state and 0 at the highest eigenstate, and for MaxCut the usual approximation ratio $\langle C \rangle / C_{\max}$. Both put every problem on the same $[0, 1]$ scale.

Model. We use one frontier model, GPT-6 Astra (OpenAI, Responses API), with its default reasoning settings and a 32,000-token output budget. Each prompt is sent 3 times (VQE, QAOA, fixed-ansatz QML) or 5 times (free-design QML) so that the spread across samples is visible; the reported mean $\pm$ standard deviation is over these repeats. Repeats are independent requests with no shared context. A second frontier model (Claude Fable 5.1) was run on a subset of the experiments with identical prompts; its traces are released but it was not run to completion and is not analysed here.

4 Experimental setup

Table 1 summarizes the tasks. All circuits are simulated exactly with a batched state-vector simulator written for this study and cross-checked against Qiskit on random circuits over the full gate set; the simulator, prompts, raw replies and analysis code are released (see Reproducibility).

QML on handwritten digits. We use the scikit-learn `digits` data set [Pedregosa et al., 2011]: 8×8 grayscale images of handwritten digits, pixel values scaled to $[0, 1]$. For each task a stratified 70/30 train/test split is fixed once; the model is shown k labelled training examples per class ($k = 5$ unless stated) rendered as images (one image per class, examples side by side), and the circuit it returns is scored on the full test split (108–109 images for a pair of digits, 540 for all ten classes). Trial t shows the same examples to both models, so the models are compared on identical prompts. In the free-design setting the model writes a circuit in a small JSON language: a number of qubits, an initial state (all-zeros, or the image amplitude-encoded on six qubits), a gate list in which every angle is an affine expression of the pixels $x[i]$ with at most one pixel per angle, and a readout that maps measured bit strings to classes (Appendix A). The *structured* condition adds three rules: at least two qubits, at least one two-qubit gate, and no two consecutive same-axis rotations on a qubit. In the fixed-ansatz setting the image is amplitude-encoded on six qubits (three qubits index the

Family	Problem	Circuit	Model writes
QML, free design	digits 3 vs 8; 4 vs 9; all 10 classes	its own (≤ 8 qubits, ≤ 200 gates)	architecture + weights
QML, free design	ablations on 3 vs 8: 1 / 10 shots; pixels as text	its own	architecture + weights
QML, structured	3 vs 8; 10 classes, with structural rules	its own (≥ 2 qubits, entangling gate)	architecture + weights
QML, fixed ansatz	3 vs 8; 10 classes	amplitude encoding + HEA, $L \in \{1, 2, 4, 8\}$	12, 18, 30 or 54 weights
VQE	transverse-field Ising chain, $N = 6$, $h \in \{0.5, 1, 2\}$	HVA $p = 2$ (4 w.); HEA $L = 2$ (18 w.)	weights
VQE, scaling	critical Ising chain, $h = 1$	HVA $p = 4$ (8 w.); HEA $L = 4$ (30 w.)	weights
VQE	Heisenberg chain, $N = 6$	HEA $L = 2$ (18 w.)	weights
VQE	H_2 molecule, 2-qubit Hamiltonian	1-parameter ansatz; HEA $L = 1$ (4 w.)	weights
QAOA	MaxCut, $n = 8$: three 3-regular, two Erdős–Rényi, one weighted ring	standard QAOA, $p \in \{1, 3\}$; $p = 6$ on two graphs	$2p$ angles

Table 1: Tasks. Every trial is one request; the model never receives a loss, gradient or accuracy.

pixel row, three the column), followed by an HEA with $L \in \{1, 2, 4, 8\}$ layers; the model writes the $6(L+1)$ rotation angles (12 to 54) and class scores are read from the first qubit (two classes) or the first four qubits (ten classes). The deeper circuits test how far hand-written weights scale: with 54 weights the ten-class circuit is a usable model for an optimizer, as Section 5 shows.

VQE. The transverse-field Ising model (TFIM) on an open chain of six spins, $H = -\sum_i Z_i Z_{i+1} - h \sum_i X_i$, is studied at $h = 0.5$ (ordered phase), $h = 1$ (critical point) and $h = 2$ (disordered phase). The Hamiltonian variational ansatz starts from $|+\rangle^{\otimes 6}$ and applies $p = 2$ layers of $rzz(\theta_{2k})$ on every bond followed by $rx(\theta_{2k+1})$ on every qubit; the hardware-efficient ansatz applies three layers of ry rotations separated by cx ladders (18 angles). At the critical point we also test a deeper HVA ($p = 4$, 8 angles) and HEA ($L = 4$, 30 angles). The antiferromagnetic Heisenberg chain $H = \sum_i (X_i X_{i+1} + Y_i Y_{i+1} + Z_i Z_{i+1})$ on six spins uses the same HEA. For H_2 we use the two-qubit Hamiltonian of O'Malley et al. [2016] near the equilibrium bond length (electronic energy, nuclear repulsion excluded), with a one-parameter ansatz that prepares $\cos(\theta/2)|10\rangle + \sin(\theta/2)|01\rangle$ from the Hartree–Fock state and with a four-angle HEA. The prompt lists every Pauli term of H with its coefficient, the circuit gate by gate, and a one-line physical description (e.g., "transverse-field Ising chain of 6 spins, open boundary, $J = 1$, $h = 0.5$"); it does not give the ground energy.

QAOA. MaxCut instances on eight nodes: three random 3-regular graphs, two Erdős–Rényi graphs with edge probability 0.5, and one ring with integer edge weights in $\{1, \ldots, 5\}$. The prompt lists the edges, states the cost $C(z) = \sum_{ij} w_{ij}[z_i \neq z_j]$, writes out the circuit for p layers with the convention $U_C(\gamma) = e^{-i\gamma C}$, $U_B(\beta) = e^{-i\beta \sum_i X_i}$, and asks for $(\gamma_1, \beta_1, \ldots, \gamma_p, \beta_p)$. We evaluate $p = 1$ and $p = 3$ on all six graphs and $p = 6$ (12 angles) on one 3-regular and one Erdős–Rényi graph.

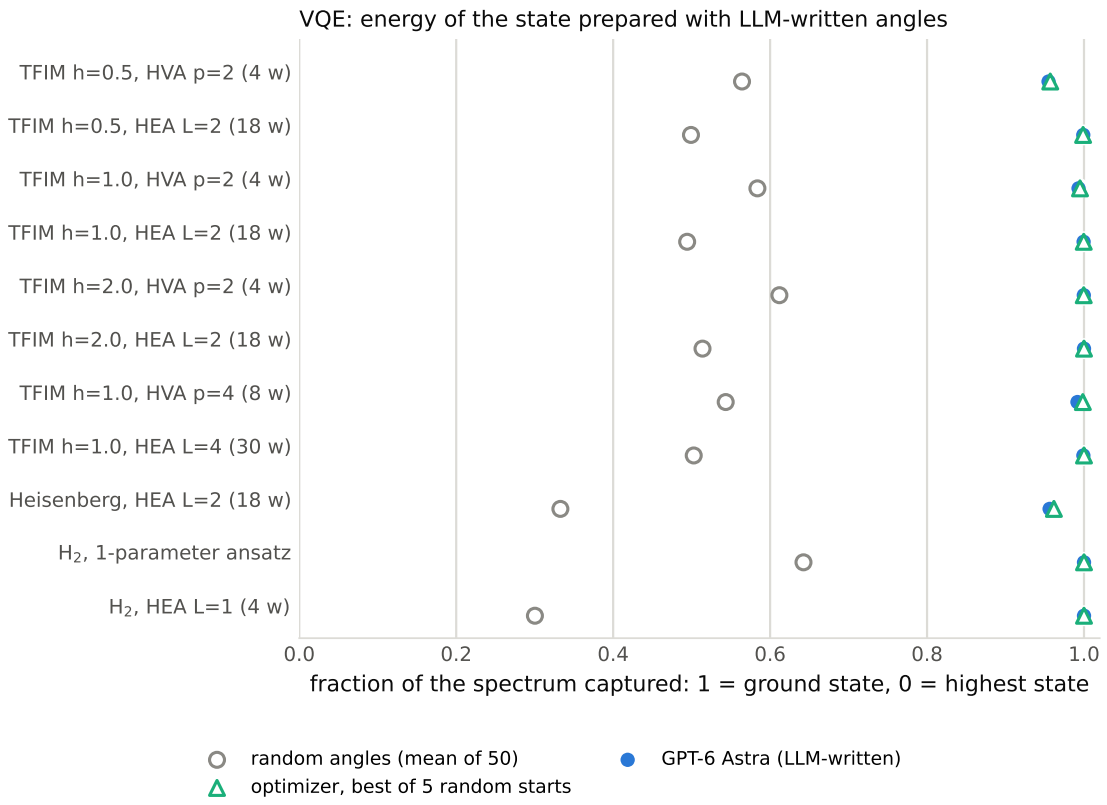


Figure 1: VQE. Fraction of the spectrum captured by the state prepared with the model's angles (1 = exact ground state), mean ± s.d. over three requests, against random angles and an optimizer with five random restarts on the same circuit.

5 Results

All 139 requests to GPT-6 Astra produced a valid circuit or weight vector on the first attempt; the repair loop was never used. Full per-condition tables are in Appendix B; this section walks through the three families.

5.1 VQE: the model writes ground-state parameters

Figure 1 and Table 4 summarize eleven problem–ansatz pairs. On every one the model's angles are within 0.007 of the optimizer on the $[0, 1]$ spectrum scale, and on seven of eleven they are within 0.001. Random angles sit at 0.30–0.64.

For the transverse-field Ising chain with the Hamiltonian variational ansatz (four angles) the model reaches 0.955, 0.993 and 1.000 at $h = 0.5, 1, 2$, against an optimizer ceiling of 0.957, 0.995 and 1.000: the ansatz itself is the limit, and the model finds its optimum. Its angles move with the field the way the physics says they should: the coupling angle of the first layer falls from 0.55 to 0.47 to 0.24 as h grows from 0.5 to 2, while the field angle rises from 1.20 to 1.36, and the three repeats of each problem agree to within 0.1 rad. With the 18-angle hardware-efficient ansatz the model reaches 0.999–1.000 on all three fields; its replies describe the target state ("a ferromagnetic state with modest canting toward $+X$, strongest at the boundaries") and set the angles to produce it. For the critical chain with a 30-angle HEA it still reaches 0.999, by setting 19–21 of the 30 angles

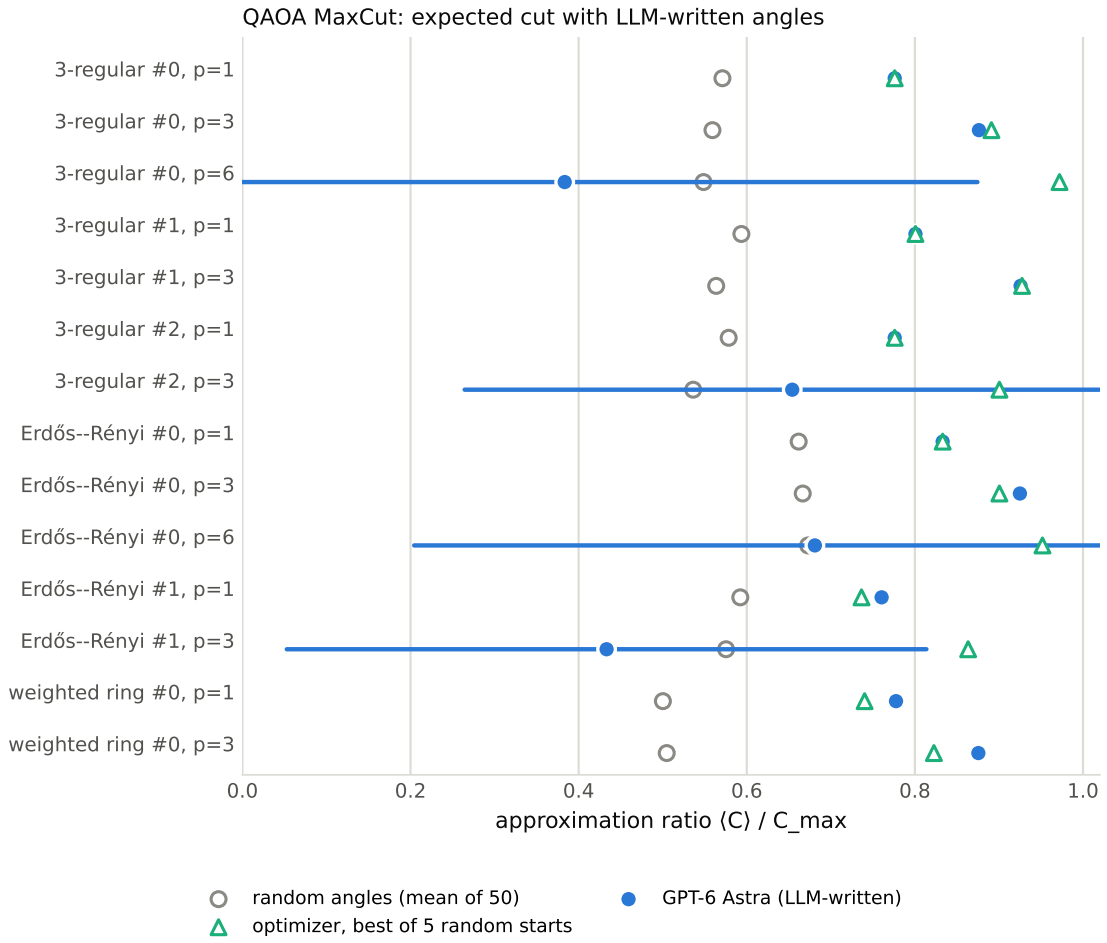


Figure 2: QAOA MaxCut on eight-node graphs. Approximation ratio $\langle C \rangle / C_{\max}$ with the model's angles, against random angles and an optimizer with five random restarts. The large spreads at $p = 3$ and $p = 6$ are sign errors on the mixing angles in 6 of 42 requests (see text).

to exactly zero and programming one rotation layer plus one correction layer; with the eight-angle HVA it reaches 0.992 against 0.999 for the optimizer, the one case where more angles made the model slightly worse rather than better.

For H_2 the model does something different: it derives the energy as a function of the single angle, $E(\theta) = -1.0520 - 0.7782 \cos\theta + 0.1820 \sin\theta$ (which is correct for the stated Hamiltonian and circuit), and returns its minimizer $\theta^* = -\arctan(0.1820/0.7782) = -0.2297$, the exact ground state. With the four-angle HEA on the same molecule it also reaches the exact energy. The largest gaps are the eight-angle HVA (0.007) and the Heisenberg chain with the HEA (0.956 vs 0.962); on the latter its reply reasons about singlet pairing on bonds but the real-amplitude HEA cannot represent the singlet ground state well with two layers, and both model and optimizer are limited by it.

5.2 QAOA: the model writes angle schedules

At depth $p = 1$ the model matches the optimizer exactly on all six graphs (0.776, 0.801, 0.776, 0.833, 0.760, 0.778, with zero variance across repeats), and on the second Erdős–Rényi graph and the weighted ring it exceeds the five-restart optimizer (0.760 vs 0.737; 0.778 vs 0.740). Its

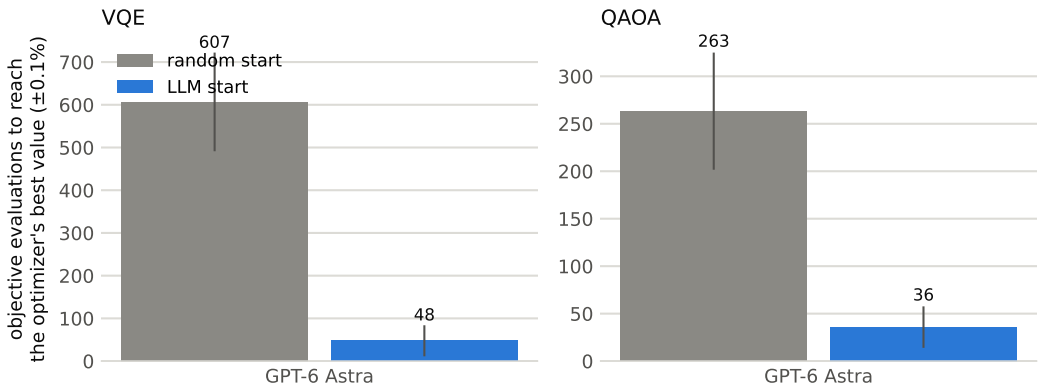


Figure 3: Loss evaluations the optimizer needs to come within 0.1% of the best value found from random starts, when started from random angles versus from the model's angles (mean ± s.e. over all VQE and QAOA requests).

method is analytic: for the first 3-regular graph it writes "the exact $p = 1$ expectation is $6 + 6\sin(4\beta)\sin(\gamma)\cos^2(\gamma) - 6\sin^2(2\beta)\sin^2(\gamma)\cos^2(\gamma)$", accounting for the graph's four triangles, and maximizes it in closed form. At $p = 3$ and $p = 6$ it writes a smooth schedule—cost angles increasing, mixing angles decreasing across layers, a pattern known from the literature on parameter concentration—and where the schedule has the right sign it is within 0.03 of the optimizer or above it (0.925 vs 0.901 on the first Erdős–Rényi graph at $p = 3$, 0.876 vs 0.823 on the weighted ring).

Six of the 42 requests failed in one specific way: every mixing angle was negated (e.g., $[0.38, -0.57, 0.65, -0.47, \ldots]$ instead of $[0.36, 0.57, 0.62, 0.47, \ldots]$). Negating β alone is not a symmetry of QAOA, and those circuits score 0.10–0.22. Re-evaluating them with $|\beta|$ gives 0.87–0.96, at the optimizer's level in every case. Over all 42 requests the mean ratio is 0.748 as written and 0.856 with the sign corrected, against 0.849 for the optimizer; over the 36 sign-correct requests it is 0.846 against 0.838. We report the as-written number as the result and the corrected one as the diagnosis.

5.3 Warm starts

Started from the model's angles, L-BFGS-B reached the target in a median of one loss evaluation in both VQE and QAOA, meaning the model's point was already within tolerance in 61% (VQE) and 64% (QAOA) of requests; the means are 48 and 36 evaluations, against 607 and 263 from random starts (Figure 3). The warm start also succeeds more often: it reached the target in 91% of VQE and 93% of QAOA runs, against 81% and 46% for random starts. The failures to reach it from the model's angles are the sign-flipped QAOA schedules and the Heisenberg chain, where the optimizer from the model's point converges to a slightly different local minimum.

5.4 QML: writing a classifier's weights

Free design. Given five examples per class and a circuit language, the model's circuits score 0.924 ± 0.032 on 3 vs 8 and 0.952 ± 0.008 on 4 vs 9 (chance 0.5), against 0.50 with random weights on the same circuits and 0.58–0.62 for an optimizer that trained the same circuits on the same ten examples (Figure 4A, Table 2). Logistic regression on the ten examples scores 0.95–0.97. The number of examples barely matters: with one example per class the model scores 0.935, with ten 0.907, and shown the pixel grids as integers instead of images 0.933. On all ten classes with three

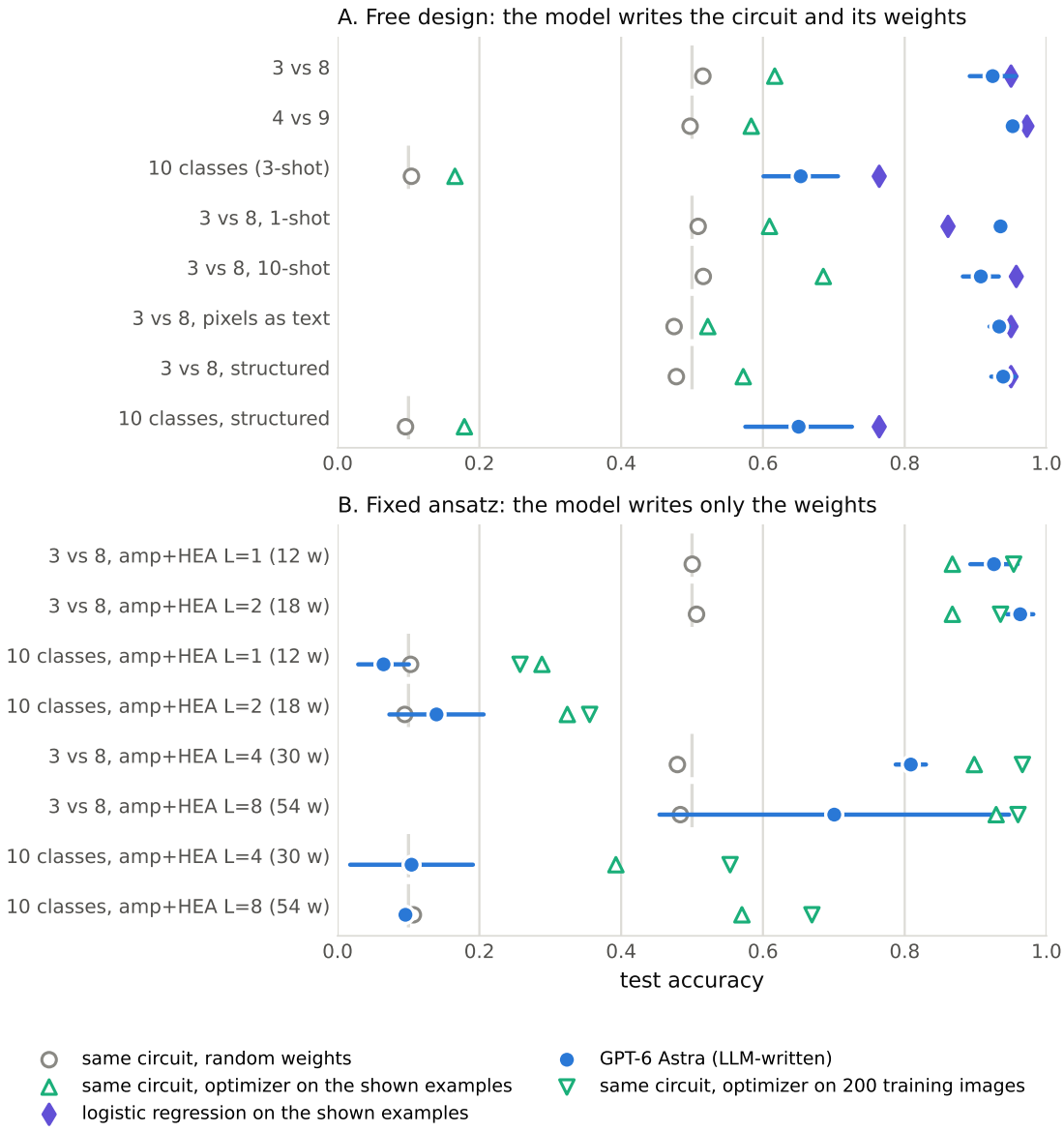


Figure 4: Digit classification. Test accuracy of the model's circuit (mean ± s.d. over five requests in A, three in B), with the same circuit under random weights, under an optimizer that saw the same examples, and logistic regression on those examples (A) or the optimizer with 200 training images (B). Gray ticks mark chance.

examples each it scores 0.653 ± 0.053 (chance 0.1), against 0.17 for the optimizer on the same thirty images and 0.76 for logistic regression.

In all 30 unconstrained requests the model designed the same kind of circuit: one qubit, a chain of 8–51 ry rotations each driven by one pixel (plus a bias rotation), and a bit-string readout. As explained in Section 6, this is a linear classifier with a smooth threshold, and the model's replies describe it as one ("positive weights collect middle-left evidence, and negative weights compensate for right-side ink"). For ten classes it scaled the same idea up: eight qubits, each a rotation chain detecting one feature ("central versus peripheral ink, upper and lower stroke balance, crossbars, and bottom closure"), read out compositionally so that, in its words, "5 combines a left-upper-stroke detector with lower-right dominance". Three of the five ten-class circuits went further and used controlled-ry gates, two of them with Toffoli gates as well, to implement that logic explicitly, scoring 0.67–0.71. Fine-tuning the model's weights with the optimizer on the shown examples did not help on any binary task (0.85–0.95 after versus 0.91–0.95 before): the optimizer, with ten examples, overfits where the model's prior does not.

Structured design. With the three structural rules (at least two qubits, at least one two-qubit gate, no consecutive same-axis rotations on a qubit) the model scores 0.939 ± 0.015 on 3 vs 8 and 0.650 ± 0.067 on ten classes, indistinguishable from the unconstrained 0.924 and 0.653, and all ten circuits are valid on the first attempt. What it built is the more interesting result. In every binary trial the model kept the linear accumulator and found a circuit identity that satisfies the rules: it interleaves z gates between the ry rotations (since $z\,ry(b)\,z = ry(-b)$, the chain still sums, with alternating signs absorbed into the weights; "alternating Z gates and signed RY angles accumulate this evidence into one decision angle", it explains), or accumulates the same weighted sum as a phase with p or rz gates between CNOTs and converts it to a probability with a final Hadamard, and then moves the result to the readout qubit with a single CNOT. For ten classes two of the five circuits are of the same kind ("the π-angle ZZ gates provide sign refocusing between successive encoding rotations"), and three use genuinely multi-qubit designs: 117 $u3$ rotations with 12 CNOTs on five qubits (0.706), and two eight-qubit circuits in which "four address qubits select a class-specific affine phase; interference converts that phase into a score monotonic in a sparse nearest-prototype discriminant", built from 39 Toffoli and 146 controlled-phase gates (0.713, the best ten-class circuit of the study). The model, in other words, knows exactly which weight space it wants to program—a signed sum of pixels—and, forbidden the obvious circuit for it, writes a different circuit that computes the same thing, or, when it must, a nearest-prototype classifier whose prototypes it writes by hand.

Fixed ansatz and scaling. When the circuit is fixed to amplitude encoding of the whole image followed by a hardware-efficient ansatz, the model has no interpretable pixel weights to write; it must reason about how rotations on the row and column qubits redistribute amplitude. For 3 vs 8 with 12 and 18 weights it does this well: 0.926 ± 0.033 and 0.963 ± 0.019, above the optimizer trained on the same examples (0.867) and close to the optimizer with 200 images (0.935–0.954). Its reply reads the circuit correctly ("3 has more ink energy in the outer row bands relative to the middle bands, while 8 is more centrally distributed; these angles implement that row-band contrast").

Figure 5 shows what happens as the circuit deepens. For 3 vs 8 the optimizer's accuracy rises monotonically with depth ($0.87 \to 0.93$ on the shown examples) while the model's falls: 0.81 ± 0.02 at 30 weights and 0.70 ± 0.25 at 54. For ten classes the circuit is too weak at 12–18 weights for anyone (the optimizer on the shown examples reaches 0.29–0.32, and 0.26–0.36 with 200 images), but with 30 and 54 weights it becomes a usable model for the optimizer (0.39 and 0.57 on the shown

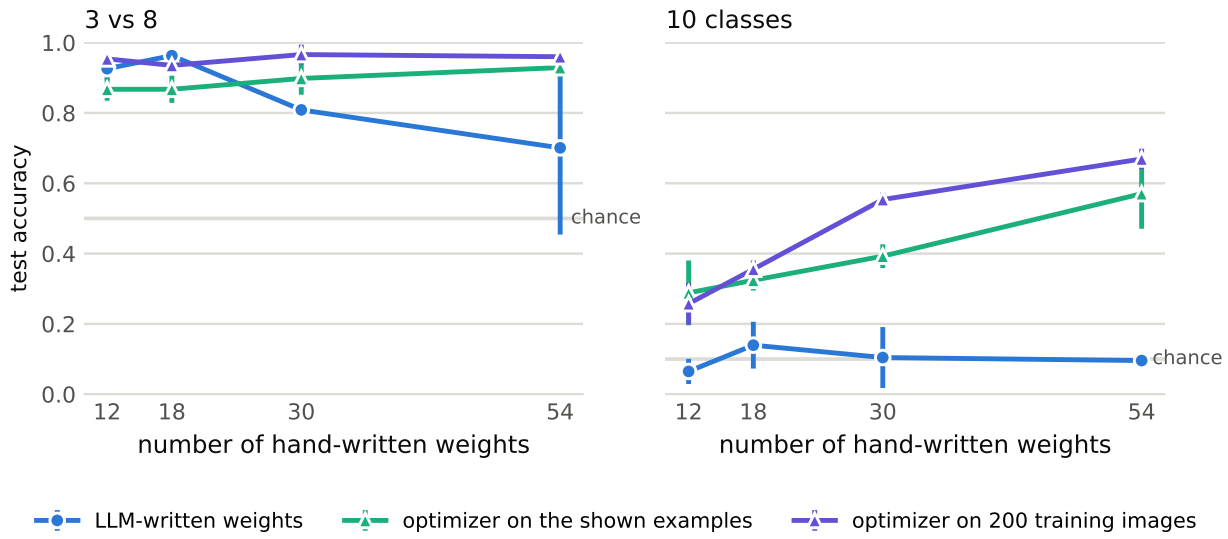


Figure 5: Scaling the fixed ansatz. Test accuracy against the number of hand-written weights (amplitude encoding + HEA with 1, 2, 4 and 8 layers) for the model, the optimizer on the shown examples, and the optimizer on 200 training images.

examples; 0.55 and 0.67 with 200 images) and the model stays at chance throughout (0.07–0.14). Its replies at depth are candid: "these are hand-designed heuristic weights, not optimized or validated weights". Started from the model's 54 weights, the optimizer reaches 0.59, so the weights are not harmful, merely uninformative.

5.5 Cost

A request cost on average 1.8k output tokens and 45 s for QAOA, 3.0k tokens and 160 s for VQE, and 5.2–5.5k tokens and 100–150 s for QML; the ten-class free-design circuits were the largest replies (14k tokens, 240 s). A single evaluation of the resulting circuit on our simulator takes milliseconds; the optimizer baselines took seconds to a minute per run.

6 Discussion

What the model actually does. Reading the replies, three strategies account for nearly all of the successes. (i) *Analytic solution.* Where the loss has a short closed form, the model derives it and solves it: for H_2 it writes $E(\theta) = -1.0520 - 0.7782\cos\theta + 0.1820\sin\theta$ and returns $\theta^* = -\arctan(0.1820/0.7782)$, which is the exact minimizer; for QAOA at $p = 1$ it writes the expectation of the cut as a trigonometric polynomial in (γ, β), triangles included, and maximizes it. (ii) *Physical interpolation.* For the Ising chain it reasons about the two limits (all spins aligned at $h \to 0$, all spins along X at $h \to \infty$) and places the angles between them according to h; its weights for the field term grow with h and those for the coupling shrink. (iii) *Geometric feature design.* For digits it names regions of the image ("middle-left ink", "bottom closure", "upper loop"), assigns each region a sign and a magnitude, and sums them in a rotation. All three are forms of reasoning about what a weight does; none is search.

Why the free-design circuits are linear classifiers. Consecutive rotations about one axis add their angles, so a chain $ry(w_1 x_{i_1})\, ry(w_2 x_{i_2}) \cdots ry(b)$ on one qubit prepares a state whose probability of reading 1 is $\sin^2\big((b + \sum_k w_k x_{i_k})/2\big)$: a linear score passed through a smooth threshold. This is the structure the model found in every unconstrained trial, and it is the natural thing to find, because it is the only circuit in the language whose weights have the meaning "how much does this pixel count". The structured condition, which forbids the chain, shows that the model can also program two-qubit circuits, but it does so by reproducing the same logic across qubits and combining the results with controlled rotations. We see this as the central lesson of the free-design experiments: the model is good at programming weights whose effect it can state in a sentence, and it chooses architectures accordingly.

Scaling. The fixed-ansatz results (Figure 5) show the limit cleanly. The amplitude-encoded circuit is a fixed unitary applied to the image vector, so each class score is the squared projection of the image onto one direction, and the weights control that direction only through a product of many rotations. At 12–18 weights the model can still say what the circuit computes ("row-band contrast", in its words) and its weights beat the optimizer; at 30–54 weights it cannot, it says so ("hand-designed heuristic weights"), and its accuracy decays toward chance while the optimizer's improves. For ten classes the circuit needs at least 54 weights before an optimizer makes it useful, and the model is at chance throughout. Weight programming therefore scales with the *interpretability* of the weight space rather than with its size: the HVA has only four to eight weights, each with a physical meaning, and the model handles them well; the 30-weight HEA for the critical Ising chain is also handled (ratio 0.999), but there the model treats most weights as zero and programs a few. Problems whose solution needs many weights with no individual meaning are beyond the method, and the model knows it. The contrast with Takehi et al. [2026], where the same model wrote networks of 10^4–10^6 weights, is instructive: those weights live in architectures the model can decompose (a convolution that detects a paddle, a layer that reads a board), so a million numbers can be written as a few hundred decisions. Fifty-four angles whose only effect is through a product of rotations on a 64-dimensional amplitude vector offer no such decomposition, and are harder to program than a million structured ones.

Warm starts. Even where the model's weights are not optimal, they are a good place to start. Across all VQE and QAOA trials the optimizer started from the model's angles reached its target after a median of one loss evaluation, that is, the model's point was already within tolerance, and in the remaining cases after tens rather than hundreds of evaluations. Since loss evaluations are runs of the quantum device, this is the most direct practical consequence of the results: a few seconds of model reasoning replaces most of the optimization loop.

Conventions and failure modes. The one systematic failure is a sign. In six QAOA trials every mixing angle came out negated; the magnitudes were right and the sign-corrected circuits match the optimizer. The prompt stated the convention $U_B(\beta) = e^{-i\beta \sum X}$ explicitly, and the gate list showed $rx(2\theta)$, so this is a reasoning slip rather than missing information, but it illustrates that weight programming is brittle in a way optimization is not: an optimizer does not care which direction is positive. In practice a single loss evaluation would catch it; we chose not to allow one, to keep the test clean.

Limitations. All circuits are small (at most eight qubits) and simulated without noise. The problems are well known, and the model has certainly seen the Ising model, QAOA on regular

graphs and the digits data set during training; part of what we measure is recall. We regard this as a feature of the method rather than a confound (a programmer's prior knowledge is the point), but it means the results do not predict performance on novel Hamiltonians or data sets. The optimizer baseline is deliberately modest (L-BFGS-B with finite differences, a few restarts, a 60-second budget) and is a reference, not a tuned upper bound; this matters most for the deep fixed-ansatz QML cases, where a stronger optimizer would widen the gap to the model. We report one model; a second frontier model (Claude Fable 5.1) ran a subset of the experiments and its traces are released with the others, but it was not run to completion and is not analysed here. Finally, our scoring never reaches the model, which is the strictest possible setting; allowing even one round of feedback would likely remove the sign failures and improve the deep circuits, and is the obvious next experiment.

7 Related work

Weight-space programming. The term and the idea are due to Takehi et al. [2026], who showed with the same model we use that a frontier LLM can read the raw weights of small networks and name the game or data they were trained on, edit 3.5% of a failing control policy's weights to make it succeed, and write from scratch, with no training and no environment rollouts, the weights of MinAtar game policies that are competitive with or better than PPO-trained ones and of a 9×9 Go player that beats smaller PPO networks; they also found that weight editing removes obstacles (a looping behaviour in a small language model) but does not add missing abilities. Our study transfers the question to variational quantum circuits. The differences in regime are what make the transfer interesting: our networks have 1–54 weights rather than thousands to millions, the weights are angles with a physical or geometric meaning, the loss is an exact expectation value rather than a game score, and the optimizer being replaced is one whose every evaluation runs a quantum device.

Initialization and warm starts for VQAs. Because VQA training is hard in general [Bittel and Kliesch, 2021] and random initializations land on barren plateaus [McClean et al., 2018], a line of work seeks good starting points: identity-block initializations [Grant et al., 2019], warm starts from relaxed classical solutions [Egger et al., 2021], and for QAOA the observation that optimal angles concentrate across instances [Brandão et al., 2018, Zhou et al., 2020] and can be tabulated for regular graphs [Wurtz and Love, 2021]. Weight-space programming can be read as initialization by a reasoning system that has absorbed this literature; our warm-start measurements quantify how much optimizer effort it saves.

Language models as optimizers and program synthesizers. Yang et al. [2024] showed that a language model can act as an optimizer when it is shown a trajectory of candidate solutions and their scores (OPRO), and FunSearch [Romera-Paredes et al., 2024] evolves programs under a scorer. Both are feedback loops. Our setting removes the loop: the model answers once, without any score, so what is measured is prior knowledge and reasoning rather than in-context optimization.

Generating weights instead of training them. Hypernetworks [Ha et al., 2017] generate the weights of one network with another, trained end to end. Our models are not trained for the task, and the "hypernetwork" is a general-purpose language model producing weights as text.

Expressibility and encodings. The capacity of a PQC depends on its layout [Sim et al., 2019] and on how data enters it [Schuld et al., 2021, Havlíček et al., 2019]. The free-design results below

illustrate this from an unusual angle: left to choose, both models pick the simplest encoding that makes their weights interpretable, which is also the one with the least quantum structure.

8 Conclusion

A frontier language model can write the parameters of a variational quantum circuit directly, with no optimizer and no feedback, whenever the parameters have a meaning it can reason about: the angles of a Hamiltonian variational ansatz, the schedule of a QAOA circuit, the pixel weights of a small classifier. In those settings its weights match a classical optimizer's, sometimes exceed an optimizer given the same data, and almost always put an optimizer within one evaluation of its best value. The approach does not extend to circuits whose weights have no individual meaning, and it is vulnerable to convention errors that an optimizer would never make. We think the right reading, in line with Takehi et al. [2026], is that language models are not optimizers but programmers, and that variational quantum circuits are a natural home for weight-space programming: their weights are few, meaningful, and expensive to train.

Reproducibility

All code, configuration files, prompts, raw model replies, parsed circuits, per-trial scores and the analysis scripts that produce every figure and table in this paper are available at `https://github.com/REPO_PLACEHOLDER`. Each trial directory holds the exact prompt (with the images shown to the model), every reply including repair turns, the circuit that was simulated, and a JSON record of all scores and baselines. `scripts/run_all.sh <model>` reruns every experiment for a model, skipping finished trials; `python -m weightprog.analysis` regenerates the figures and tables. The simulator is verified against Qiskit by `pytest`. Model replies are stochastic, so a rerun will not reproduce our numbers exactly; the released traces are the record of the runs reported here.

References

Lennart Bittel and Martin Kliesch. Training variational quantum algorithms is NP-hard. *Physical Review Letters*, 127(12):120502, 2021.

Fernando GSL Brandão, Michael Broughton, Edward Farhi, Sam Gutmann, and Hartmut Neven. For fixed control parameters the quantum approximate optimization algorithm's objective function value concentrates for typical instances. *arXiv preprint arXiv:1812.04170*, 2018.

Marco Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, and Patrick J Coles. Variational quantum algorithms. *Nature Reviews Physics*, 3(9):625–644, 2021.

Daniel J Egger, Jakub Mareček, and Stefan Woerner. Warm-starting quantum optimization. *Quantum*, 5:479, 2021.

Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028*, 2014.

Edward Grant, Leonard Wossnig, Mateusz Ostaszewski, and Marcello Benedetti. An initialization strategy for addressing barren plateaus in parametrized quantum circuits. *Quantum*, 3:214, 2019.

David Ha, Andrew Dai, and Quoc V Le. Hypernetworks. In *International Conference on Learning Representations*, 2017.

Vojtěch Havlíček, Antonio D Córcoles, Kristan Temme, Aram W Harrow, Abhinav Kandala, Jerry M Chow, and Jay M Gambetta. Supervised learning with quantum-enhanced feature spaces. *Nature*, 567(7747):209–212, 2019.

Abhinav Kandala, Antonio Mezzacapo, Kristan Temme, Maika Takita, Markus Brink, Jerry M Chow, and Jay M Gambetta. Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets. *Nature*, 549(7671):242–246, 2017.

Jarrod R McClean, Sergio Boixo, Vadim N Smelyanskiy, Ryan Babbush, and Hartmut Neven. Barren plateaus in quantum neural network training landscapes. *Nature Communications*, 9:4812, 2018.

Kosuke Mitarai, Makoto Negoro, Masahiro Kitagawa, and Keisuke Fujii. Quantum circuit learning. *Physical Review A*, 98(3):032309, 2018.

Peter JJ O'Malley, Ryan Babbush, Ian D Kivlichan, Jonathan Romero, Jarrod R McClean, Rami Barends, Julian Kelly, Pedram Roushan, Andrew Tranter, Nan Ding, et al. Scalable quantum simulation of molecular energies. *Physical Review X*, 6(3):031007, 2016.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Adrián Pérez-Salinas, Alba Cervera-Lierta, Elies Gil-Fuster, and José I Latorre. Data re-uploading for a universal quantum classifier. *Quantum*, 4:226, 2020.

Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J Love, Alán Aspuru-Guzik, and Jeremy L O'Brien. A variational eigenvalue solver on a photonic quantum processor. *Nature Communications*, 5:4213, 2014.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.

Maria Schuld, Alex Bocharov, Krysta M Svore, and Nathan Wiebe. Circuit-centric quantum classifiers. *Physical Review A*, 101(3):032308, 2020.

Maria Schuld, Ryan Sweke, and Johannes Jakob Meyer. Effect of data encoding on the expressive power of variational quantum-machine-learning models. *Physical Review A*, 103(3):032430, 2021.

Sukin Sim, Peter D Johnson, and Alán Aspuru-Guzik. Expressibility and entangling capability of parameterized quantum circuits for hybrid quantum-classical algorithms. *Advanced Quantum Technologies*, 2(12):1900070, 2019.

Rikiya Takehi, Jayden Teoh, Alex Zhang, and Omar Khattab. Sparks of weight-space programming: LLMs can natively read & write neural network weights. Blog post, MIT CSAIL, November 2026. URL `https://rikiyat.github.io/blog/weight-space-programming/`.

Dave Wecker, Matthew B Hastings, and Matthias Troyer. Progress towards practical quantum variational algorithms. *Physical Review A*, 92(4):042303, 2015.

Roeland Wiersema, Cunlu Zhou, Yvette de Sereville, Juan Felipe Carrasquilla, Yong Baek Kim, and Henry Yuen. Exploring entanglement and optimization within the hamiltonian variational ansatz. *PRX Quantum*, 1(2):020319, 2020.

Jonathan Wurtz and Peter Love. Fixed-angle conjectures for the quantum approximate optimization algorithm on regular maxcut graphs. *Physical Review A*, 104(5):052419, 2021.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations*, 2024.

Leo Zhou, Sheng-Tao Wang, Soonwon Choi, Hannes Pichler, and Mikhail D Lukin. Quantum approximate optimization algorithm: Performance, mechanism, and implementation on near-term devices. *Physical Review X*, 10(2):021067, 2020.

A The circuit language for free design

In the free-design QML experiments the model writes a JSON object with four fields. `n_qubits` is the register size. `state_prep` is either `{"type": "zero"}` or `{"type": "amplitude", "qubits": [q0, ..., q5]}`, which loads the normalized 64-pixel vector as the amplitudes of the six listed qubits (the first three index the row, the last three the column). `gates` is an ordered list; each entry names a gate from {h, x, y, z, s, sdg, t, tdg, sx, rx, ry, rz, p, u3, cx, cy, cz, swap, crx, cry, crz, cp, rxx, ryy, rzz, ccx}, its qubits, and for rotations an angle written as a number or as a string expression over numbers, `pi`, pixels `x[i]`, and $+ - * /$. Angles must be affine in the pixels and, in the default condition, contain at most one pixel. `readout` is either `bitstrings` (a list of measured qubits and, per class, the bit strings whose total probability is the class score) or `qubit_per_class` (the class score is the probability that its qubit reads 1). The predicted class is the one with the highest score. The complete system prompt, including these rules, is in the repository (`weightprog/prompts.py`) and in every trial directory as `prompt.txt`.

B Full result tables

Tables 2–5 list every condition: validity (usable circuits / requests), the score of the model-written weights, the random-weight floor, the optimizer from random starts, and the optimizer started from the model's weights. Means ± standard deviations are over repeated requests.

Task	Model	Valid	LLM	Random w.	Optimizer	LLM→opt.	LogReg
3 vs 8	GPT-6 Astra	5/5	0.924 ± 0.032	0.515 ± 0.018	0.617 ± 0.094	0.869 ± 0.094	0.950 ± 0.024
4 vs 9	GPT-6 Astra	5/5	0.952 ± 0.008	0.497 ± 0.013	0.583 ± 0.097	0.950 ± 0.026	0.972 ± 0.015
10 classes (3-shot)	GPT-6 Astra	5/5	0.653 ± 0.053	0.104 ± 0.007	0.166 ± 0.082	0.636 ± 0.045	0.764 ± 0.016
3 vs 8, 1-shot	GPT-6 Astra	5/5	0.935 ± 0.007	0.509 ± 0.009	0.609 ± 0.107	0.926 ± 0.025	0.861 ± 0.066
3 vs 8, 10-shot	GPT-6 Astra	5/5	0.907 ± 0.025	0.516 ± 0.040	0.685 ± 0.164	0.846 ± 0.057	0.957 ± 0.011
3 vs 8, pixels as text	GPT-6 Astra	5/5	0.933 ± 0.014	0.475 ± 0.028	0.522 ± 0.063	0.863 ± 0.081	0.950 ± 0.024
3 vs 8, structured	GPT-6 Astra	5/5	0.939 ± 0.017	0.478 ± 0.014	0.572 ± 0.060	0.898 ± 0.048	0.950 ± 0.024
10 classes, structured	GPT-6 Astra	5/5	0.650 ± 0.075	0.096 ± 0.004	0.179 ± 0.077	0.529 ± 0.102	0.764 ± 0.016

Table 2: Free-design QML, test accuracy.

Task, ansatz	Model	Valid	LLM	Random w.	Opt. (shown)	LLM→opt.	Opt. (200 img.)
3 vs 8, amp+HEA L=1 (12 w)	GPT-6 Astra	3/3	0.926 ± 0.033	0.500 ± 0.023	0.867 ± 0.033	0.867 ± 0.033	0.954 ± 0.000
3 vs 8, amp+HEA L=2 (18 w)	GPT-6 Astra	3/3	0.963 ± 0.019	0.506 ± 0.018	0.867 ± 0.039	0.867 ± 0.039	0.935 ± 0.000
10 classes, amp+HEA L=1 (12 w)	GPT-6 Astra	3/3	0.065 ± 0.036	0.103 ± 0.009	0.288 ± 0.092	0.243 ± 0.076	0.257 ± 0.059
10 classes, amp+HEA L=2 (18 w)	GPT-6 Astra	3/3	0.140 ± 0.066	0.095 ± 0.008	0.324 ± 0.029	0.246 ± 0.023	0.356 ± 0.025
3 vs 8, amp+HEA L=4 (30 w)	GPT-6 Astra	3/3	0.809 ± 0.021	0.479 ± 0.054	0.898 ± 0.046	0.920 ± 0.075	0.966 ± 0.028
3 vs 8, amp+HEA L=8 (54 w)	GPT-6 Astra	3/3	0.701 ± 0.247	0.483 ± 0.049	0.929 ± 0.028	0.938 ± 0.023	0.960 ± 0.005
10 classes, amp+HEA L=4 (30 w)	GPT-6 Astra	3/3	0.104 ± 0.087	0.102 ± 0.004	0.393 ± 0.034	0.408 ± 0.044	0.554 ± 0.020
10 classes, amp+HEA L=8 (54 w)	GPT-6 Astra	3/3	0.096 ± 0.006	0.107 ± 0.006	0.570 ± 0.100	0.594 ± 0.024	0.669 ± 0.029

Table 3: Fixed-ansatz QML, test accuracy.

Problem, ansatz	Model	Valid	LLM	Random	Optimizer	LLM→opt.
TFIM h=0.5, HVA p=2 (4 w)	GPT-6 Astra	3/3	0.955 ± 0.001	0.564 ± 0.026	0.957 ± 0.000	0.957 ± 0.000
TFIM h=0.5, HEA L=2 (18 w)	GPT-6 Astra	3/3	0.999 ± 0.001	0.499 ± 0.026	0.999 ± 0.000	0.999 ± 0.001
TFIM h=1.0, HVA p=2 (4 w)	GPT-6 Astra	3/3	0.993 ± 0.001	0.584 ± 0.004	0.995 ± 0.000	0.995 ± 0.000
TFIM h=1.0, HEA L=2 (18 w)	GPT-6 Astra	3/3	0.999 ± 0.000	0.494 ± 0.012	1.000 ± 0.000	1.000 ± 0.000
TFIM h=2.0, HVA p=2 (4 w)	GPT-6 Astra	3/3	1.000 ± 0.000	0.612 ± 0.014	1.000 ± 0.000	1.000 ± 0.000
TFIM h=2.0, HEA L=2 (18 w)	GPT-6 Astra	3/3	1.000 ± 0.000	0.514 ± 0.008	1.000 ± 0.000	1.000 ± 0.000
TFIM h=1.0, HVA p=4 (8 w)	GPT-6 Astra	3/3	0.992 ± 0.003	0.543 ± 0.019	0.999 ± 0.000	0.999 ± 0.000
TFIM h=1.0, HEA L=4 (30 w)	GPT-6 Astra	3/3	0.999 ± 0.001	0.503 ± 0.011	1.000 ± 0.000	1.000 ± 0.000
Heisenberg, HEA L=2 (18 w)	GPT-6 Astra	3/3	0.956 ± 0.005	0.333 ± 0.016	0.962 ± 0.000	0.959 ± 0.000
H_2, 1-parameter ansatz	GPT-6 Astra	3/3	1.000 ± 0.000	0.643 ± 0.033	1.000 ± 0.000	1.000 ± 0.000
H_2, HEA L=1 (4 w)	GPT-6 Astra	3/3	1.000 ± 0.000	0.300 ± 0.034	1.000 ± 0.000	1.000 ± 0.000

Table 4: VQE, fraction of the spectrum captured (1 = exact ground state).

Graph, depth	Model	Valid	LLM	Random	Optimizer	LLM→opt.
3-regular #0, p=1	GPT-6 Astra	3/3	0.776 ± 0.000	0.571 ± 0.020	0.776 ± 0.000	0.776 ± 0.000
3-regular #0, p=3	GPT-6 Astra	3/3	0.876 ± 0.005	0.559 ± 0.015	0.891 ± 0.016	0.901 ± 0.000
3-regular #0, p=6	GPT-6 Astra	3/3	0.383 ± 0.491	0.549 ± 0.018	0.972 ± 0.010	0.979 ± 0.012
3-regular #1, p=1	GPT-6 Astra	3/3	0.801 ± 0.000	0.594 ± 0.006	0.801 ± 0.000	0.801 ± 0.000
3-regular #1, p=3	GPT-6 Astra	3/3	0.926 ± 0.004	0.564 ± 0.014	0.927 ± 0.015	0.936 ± 0.000
3-regular #2, p=1	GPT-6 Astra	3/3	0.776 ± 0.000	0.578 ± 0.014	0.776 ± 0.000	0.776 ± 0.000
3-regular #2, p=3	GPT-6 Astra	3/3	0.654 ± 0.390	0.536 ± 0.008	0.901 ± 0.000	0.891 ± 0.016
Erdős–Rényi #0, p=1	GPT-6 Astra	3/3	0.833 ± 0.000	0.662 ± 0.007	0.833 ± 0.000	0.833 ± 0.000
Erdős–Rényi #0, p=3	GPT-6 Astra	3/3	0.925 ± 0.002	0.667 ± 0.005	0.901 ± 0.032	0.934 ± 0.000
Erdős–Rényi #0, p=6	GPT-6 Astra	3/3	0.681 ± 0.477	0.673 ± 0.003	0.952 ± 0.006	0.965 ± 0.023
Erdős–Rényi #1, p=1	GPT-6 Astra	3/3	0.760 ± 0.000	0.592 ± 0.014	0.737 ± 0.041	0.760 ± 0.000
Erdős–Rényi #1, p=3	GPT-6 Astra	3/3	0.433 ± 0.380	0.575 ± 0.014	0.863 ± 0.033	0.882 ± 0.000
weighted ring #0, p=1	GPT-6 Astra	3/3	0.778 ± 0.000	0.500 ± 0.011	0.740 ± 0.065	0.778 ± 0.000
weighted ring #0, p=3	GPT-6 Astra	3/3	0.876 ± 0.004	0.505 ± 0.024	0.823 ± 0.019	0.893 ± 0.000

Table 5: QAOA MaxCut, approximation ratio.